DynCodec: Codec-Compatible Compression of Dynamic Neural Scene Representations

Jin Zhou
George Mason University
Fairfax, Virginia, USA

Na Li
Rutgers University
Piscataway, New Jersey, USA

Mufeng Zhu
Rutgers University
Piscataway, New Jersey, USA

Songqing Chen
George Mason University
Fairfax, Virginia, USA

Yao Liu
Rutgers University
Piscataway, New Jersey, USA

Abstract

Efficient delivery of dynamic neural scene representations is essential for emerging multimedia applications such as immersive streaming and real-time VR/AR. We present DynCodec, a codec-aware neural framework that enables efficient transmission and reconstruction of dynamic scenes by reformulating dynamic neural representations as a residual signal over a shared static base. Built on Tri-MipRF for static scene modeling, DynCodec adopts the first frame as a high-quality reference and encodes subsequent frames with a compact, factorized representation of temporal residuals. By appropriately leveraging residual modeling, feature factorization, and codec-based compression, DynCodec significantly reduces representation size while maintaining temporal consistency and rendering fidelity. Experiments on both synthetic and real-world datasets demonstrate that our method achieves an effective trade-off between storage reduction and PSNR under practical codec configurations. These results indicate that DynCodec provides a practical and scalable solution for bandwidth-limited and resource-constrained scenarios such as dynamic scene delivery and edge-based immersive applications.

Keywords

Neural Radiance Fields (NeRF); Model Compression; Dynamic Scene Reconstruction

ImmerCom '26, Texas, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM Reference Format:
Jin Zhou, Na Li, Mufeng Zhu, Songqing Chen, and Yao Liu. 2026. DynCodec: Codec-Compatible Compression of Dynamic Neural Scene Representations. In . ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Neural Radiance Fields (NeRF) have revolutionized novel view synthesis by enabling high-fidelity 3D scene reconstruction from sparse 2D images [9]. By employing a deep neural network to encode volumetric scene representations, NeRF facilitates highly realistic novel view synthesis. Traditional NeRF methods require substantial training time and memory and often produce aliasing artifacts in complex scenes [14]. These issues are amplified in dynamic settings, where efficient adaptation is critical.

In this paper, we propose DynCodec, a novel codec-compatible framework for efficiently compressing dynamic scene reconstruction. DynCodec builds on a Tri-MipRF [6] backbone and represents a dynamic scene as a static reference with compact, factorized temporal residuals. This representation explicitly separates static content from dynamic changes, making the features more suitable for video codec compression. It is designed to be compatible with standard video codecs, allowing effective reduction of inter-frame redundancy. By aligning the representation with standard video codec mechanisms, DynCodec effectively reduces inter-frame redundancy while maintaining rendering quality. As a result, DynCodec achieves high compression ratios with competitive visual fidelity. Our contributions are summarized as follows: First, we propose DynCodec, a new framework that combines a factorized representation of the temporal residual with a Tri-MipRF baseline and video-codec compression for dynamic NeRF scenes. Second, we introduce a quality-aware feature design that allows flexible trade-offs between visual fidelity and storage via quantization parameters. Third, we demonstrate through extensive experiments on synthetic and real-world datasets that DynCodec achieves good compression

performance while remaining competitive in rendering quality. DynCodec supports deployment under different bandwidth and hardware constraints, providing a good balance between memory efficiency and visual quality for dynamic NeRF applications.

This paper is organized as follows. Section 2 provides background on NeRF-based scene modeling. Section 3 details our proposed approach. Section 4 discusses experimental results, and Section 5 concludes the paper with insights and future research directions.

2 Related Work

Neural Radiance Fields (NeRF) [9] represents a significant advancement in 3D scene reconstruction, leveraging volumetric rendering and implicit neural networks to produce highly realistic renderings from sparse input images. However, the high computational cost of training and rendering remains a significant challenge. Several extensions address these limitations. NeRF−−[17] learns camera poses and intrinsics directly from RGB images, enabling novel view synthesis without pre-calibrated camera information. To improve rendering quality, Mip-NeRF [1] introduces a continuous multiscale representation that models pixel samples as conical frustums and applies integrated positional encoding, enabling scale-aware rendering and reducing aliasing artifacts. Tri-MipRF [6] introduces a tri-plane mipmapped representation, enabling multi-resolution scene encoding that enhances both efficiency and scalability. Methods such as Instant-NGP [10], KiloNeRF [12], and PlenOctrees [19] improve rendering efficiency by redesigning the scene representation. Instant-NGP uses multiresolution hash encoding to enable a smaller MLP, reducing computational cost and accelerating training and inference.

2.1 Tri-Plane Representation in Neural Radiance Fields

Tri-plane representations have emerged as a memory-efficient alternative to volumetric grids in NeRFs, encoding 3D features onto three orthogonal 2D planes. This format enables efficient feature querying, reduced memory usage, and scalable high-fidelity rendering. TensoRF [3] factorizes the radiance field into a set of low-rank tensor components, enabling highly efficient and compact scene encoding. At the same time, EG3D [2] introduces tri-plane grids in GAN-based 3D generation, offering high-quality rendering with low overhead.

Despite acceleration techniques, neural radiance fields can still require substantial memory, particularly for large scenes and high-resolution representations, limiting their deployment in resource-constrained environments.

To address the trade-off between rendering quality and efficiency, Tri-MipRF [6] decomposes the 3D feature space into three orthogonal mipmapped planes with efficient cone casting. This design enables anti-aliased, high-fidelity rendering across varying viewing scales.

2.2 Neural Radiance Fields in Dynamic Scenarios

Traditional Neural Radiance Fields (NeRF) have demonstrated strong performance in reconstructing static scenes. In NeRF, rays are traced from the camera position ($\mathbf{o} = (x_o, y_o, z_o)$) through each pixel in the rendered image. If the direction of a ray is $\mathbf{d}$, the samples on this ray can be denoted as $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, and Given N samples along the ray, the color of ray $\mathbf{r}$ can be estimated as:

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^{N} T_i(1 - \exp(-\sigma_i\delta_i))\mathbf{c}_i, \text{where } T_i = \exp(-\sum_{j=1}^{i-1} \sigma_j\delta_j)$$

where T_i is the transmittance at sample i, σ_i and $\mathbf{c}_i$ are the volume density and color (e.g., in (R, G, B) representation) at sample i. $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples along the ray. Applying NeRF to dynamic scenarios requires effective spatiotemporal modeling, robust motion representation, and memory-efficient network architectures and optimization strategies. To address these challenges, several methods have been developed to enable more effective reconstruction of dynamic scenes while balancing computational efficiency and visual fidelity. One of the most significant approaches for modeling dynamic NeRFs is D-NeRF [11], which extends the original NeRF by introducing a deformation network that learns time-dependent deformation fields to map space-time coordinates into a canonical space, where a NeRF-like radiance field is trained. However, D-NeRF remains computationally expensive, limiting real-time use. Another important method, DyNeRF [8], models dynamics using a compact set of temporal latent codes that condition a time-varying radiance field. It enables smooth and temporally consistent 3D videos from multi-view sequences and realistic synthesis of complex time-varying scenes. NR-NeRF [16] introduced a deformation model for dynamic, non-rigid scenes from monocular video by learning per-image latent codes. However, it is still limited by the computational speed. Recent advances, such as K-Planes [4] and HexPlane [7], extend the idea of structured feature planes to dynamic neural radiance fields by factorizing spatiotemporal information into low-dimensional 2D feature planes. These methods demonstrate how plane-based factorization can scale NeRF-based rendering to memory- and computation-efficient dynamic scene reconstruction.

Despite these advances, many methods focus on static scenes or rely on per-scene optimization to train compact

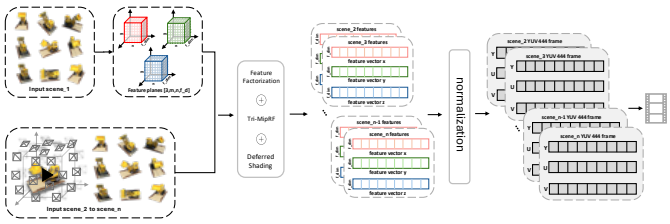

Figure 1: Pipeline of the proposed DynCodec framework for dynamic scene reconstruction.

representations whose storage characteristics are largely determined by the representation and training configuration. Our proposed DynCodec compresses the learned temporal feature maps using a video codec, enabling different bitrate-quality tradeoffs through the quantization parameter (QP) without retraining the neural representation.

3 Methodology

This section presents our approach, named DynCodec. Fig 1 shows the overall pipeline of DynCodec for dynamic scene reconstruction. Our approach encodes frame-wise residuals from a reference frame. Designed for efficient processing of dynamic scenes, DynCodec integrates feature factorization with optimization and compression techniques to improve storage efficiency and enhance reconstruction quality. Given a dynamic scene with n frames, the process is divided into three main stages: ① The first scene (i.e., the initial frame of the sequence) is fully trained using the original Tri-MipRF [6] module. The resulting tri-plane features are saved as the reference for subsequent frames. ② Each of the remaining $n-1$ scenes is trained independently using the DynCodec framework. Each frame is represented using compact, factorized temporal residuals rather than full features, which reduces storage while preserving reconstruction quality. ③ The generated feature vectors are quantized into 8-bit values that can be interpreted as grayscale intensities. These quantized features are then converted into YUV frames and concatenated into a feature video, enabling efficient storage and transmission of dynamic scene content. This section describes each stage of DynCodec in more detail.

3.1 Compact Feature Parameterization

DynCodec is a framework built upon Tri-MipRF [6], a method for efficient, high-quality reconstruction of static scenes. Tri-MipRF represents a scene using three orthogonal mipmaps, allowing high-fidelity, anti-aliased renderings. However, when applied to dynamic scenarios, Tri-MipRF faces significant challenges due to the increased complexity of handling multiple frames over time. The key challenges include managing storage requirements and maintaining high re-rendering quality, both of which are crucial for dynamic scenes processing.

In dynamic scenes, processing multiple frames substantially increases the data size and computational demands. For example, if inference for a single scene requires m MB of data, and a dynamic scene consists of n frames, the total data size for transmission becomes $n \times m$ MB. As video length or scene complexity increases, data requirements expand rapidly, leading to significant transmission and processing constraints. To mitigate these challenges, proposed methods such as K-Planes [4] exploit spatial and temporal structures through factorized scene representations. K-Planes jointly represents a dynamic scene by decomposing its 4D space into spatial and space-time feature planes, reducing memory usage and enabling efficient optimization. Although these approaches improve representation efficiency, balancing the trade-off between transmission cost and visual quality remains challenging for dynamic scenes. To overcome these limitations, our proposed approach, DynCodec, introduces a Compact Parameterization of the dynamic components. The main idea is to represent a dynamic scene as a high-quality static baseline plus a compact, learnable **temporal residual**. This ensures effective reconstruction even in resource-constrained environments by only storing the small set of parameters that describe the change between frames. We integrate this into the scene representation learning as follows. First, for the first frame ($n = 1$) of a dynamic sequence, we represent the scene as $\mathcal{M}_1$, consisting of three feature planes trained using Tri-MipRF: $\mathcal{M}_1 = \{\mathcal{F}_{xy}, \mathcal{F}_{xz}, \mathcal{F}_{yz}\}_{base}$. Each plane is a tensor of dimensions `[H, W, C]`, where `H` and `W` are spatial resolutions and `C` is the feature dimension. The three feature planes are kept frozen for subsequent frames. For subsequent frame n ($n > 1$), the representation $\mathcal{M}_n$ consists of both the baseline model (i.e., $\mathcal{M}_1$) and a compact, factorized representation of the temporal residual. This representation consists of three trainable feature vectors, which we denote as $\{\mathcal{V}_x, \mathcal{V}_y, \mathcal{V}_z\}_n$ for scene n. Each vector is a tensor with dimensions `[1, C, L, 1]`, where `L` is the length. To generate the feature vector for an arbitrary point $p = (x, y, z)$ in a dynamic frame $n > 1$, we query the representation $\mathcal{M}_n$. For clarity, we use lowercase letters to denote the feature vectors that result from sampling the corresponding uppercase tri-plane feature planes and feature vectors at the coordinates of point p.

First, residual feature $f_{\text{residual}}(p)$ is reconstructed by sampling from the feature vectors and performing an element-wise product of the resulting vectors: $f_{\text{residual}}(p) = v_x(p) \odot v_y(p) \odot v_z(p)$. The final feature vector of point p is then obtained by concatenating the three feature vectors sampled from the base feature planes with the single, calculated residual feature vector: $f(p) = f_{xy}(p) \| f_{xz}(p) \| f_{yz}(p) \| f_{\text{residual}}(p)$

where || represents concatenation along the feature dimension. This combined feature vector, $f(p)$, is then passed to a small MLP for the color and density prediction. By applying feature factorization during representation learning, the model significantly reduces storage and transmission requirements while maintaining essential features for dynamic scene reconstruction. However, despite using reference feature planes to enhance re-rendering quality, some performance loss remains. Further optimization is necessary to improve visual quality, which directly impacts the overall user experience.

To address this challenge, we integrate deferred shading into the re-rendering process to reduce computational overhead. By separating geometry processing from lighting calculations, deferred shading avoids repeated computations and enables more efficient rendering. After incorporating deferred shading into DynCodec, we observe improved rendering speed, making the re-rendering process more efficient for dynamic scenes.

Since DynCodec is designed to enable efficient reconstruction and re-rendering of dynamic scenes across different computational environments, it employs an adaptive re-rendering approach to support a wide range of devices. This adaptability is achieved by adjusting feature dimensions using a "feature dimension factor", allowing the system to optimize the balance between visual quality and computational efficiency based on specific hardware capabilities. For instance, when the feature dimension factor is set to 1, donated as "FD_1", the feature vector shape is `[1,C,L,1]`, where the residual feature length matches the triplane feature length `C`. Increasing the feature dimension factor to 3 results in a feature vector shape of `[1,3×C,L,1]`, named as "FD_3", where the residual feature length is three times the triplane feature length. This scalability allows for higher-fidelity rendering on devices with greater computational capacity. By offering this flexibility, DynCodec efficiently operates across diverse hardware configurations while maintaining an optimal balance between performance and visual quality.

3.2 Codec-based Feature Compression

After completing the training process using the DynCodec framework, each scene is represented by three feature vectors, $\{\mathcal{V}_x, \mathcal{V}_y, \mathcal{V}_z\}_n$, along with the specified feature dimensions. However, a key challenge arises as the storage requirements for the entire dynamic scene increase significantly with the length of the dynamic video. To address this issue and enhance storage efficiency while preserving re-rendering quality, we employ a normalization method combined with video codec techniques from an existing work [20]. This approach improves feature representation, reducing storage demands while maintaining the high-fidelity reconstruction of dynamic scenes.

To achieve high compression efficiency, first normalize the 32-bit floating-point feature values, then quantize them to 8-bit integers, which can be interpreted as pixel values ranging from 0 to 255 in grayscale. The parameters used for normalization are stored in a dictionary and used to restore the decoded grayscale values to their original floating-point scale. Following normalization, the feature vectors are mapped into three channels (Y, U, and V), forming a YUV444 frame with the resolution [n×C, L], where n is the feature dimension factor. For instance, the feature vector $\mathcal{V}_x$ is mapped to channel Y. These frames are then concatenated and compressed into a single feature video using FFmpeg [15], which significantly reduces storage requirements while preserving essential information for re-rendering.

Moreover, adjusting the video codec's quantization parameter (QP) configurations enables adaptive compression, allowing a flexible trade-off between storage efficiency and reconstruction quality. To improve compression while maintaining rendering quality, we adopt maximum occupancy map for each scene. Since frames in the same scene have similar occupancy patterns, this shared map covers the occupied regions across all frames and avoids storing a separate map for each frame. Building on this technique, we further optimize the MLP by normalizing its parameters and applying the same compression strategy using FFmpeg. To minimize the impact of compression on the MLP network parameters, we limit compression to QP_0 and compress the MLP network individually for each frame, ensuring efficient storage while preserving high fidelity. As a result, a single compressed video can store the feature vectors for multiple frames in a dynamic scene, making it a simple and efficient way to manage dynamic scene content.

4 Performance Evaluation

We evaluate DynCodec across multiple key dimensions, focusing on reconstruction quality and compression efficiency. Our experiments are conducted on both synthetic and real-world datasets, where we benchmark two distinct feature configurations across a range of quantization parameters. Furthermore, we provide a comprehensive ablation analysis to quantify the individual contributions of our framework's core components to the overall system performance.

4.1 Datasets and Experiments setup

Datasets. For our experiments, we evaluate the performance of DynCodec using multiple dynamic datasets and compare it with state-of-the-art (SOTA) methods. The first dataset we use is from [21], which contains synthetic sequences featuring various objects and motions including Lego, Pig,

Worker, and Amily. These sequences were generated using Blender and are also available in *png* images, making them suitable for dynamic scene reconstruction evaluation. We also use DyNeRF [8], one of the most widely adopted datasets for dynamic neural radiance field research. DyNeRF is a multi-view video dataset designed to support dynamic NeRF-based studies.

Experiments setup. DynCodec has been implemented using PyTorch, using FFmpeg as the compression tool. To benchmark its performance, we compare DynCodec against leading SOTA methods, including K-Planes [4] and NeRFPlayer [13]. The evaluation focuses on rendering quality and model size, where Peak Signal-to-Noise Ratio (PSNR) [5] and SSIM [5] is used to measure rendering quality, and the model size is used for evaluating storage efficiency.

4.2 Results

In this section, we evaluate DynCodec in terms of reconstruction quality and compression efficiency on dynamic scenes. We examine whether DynCodec can reduce model size while preserving rendering fidelity under practical compression settings.

Table 1: Comparison of PSNR, SSIM, and model size across frames on the DyNeRF dataset. "Model Size" represents the averaged per-frame model size in KB.

Method	PSNR	SSIM	Model Size
NeRFPlayer	32.05	0.938	2427
K-Planes	30.22	0.925	539
DynCodec(QP_10, FD_1)	29.76	0.898	121.92
DynCodec(QP_20, FD_1)	29.50	0.896	118.48
DynCodec(QP_10, FD_3)	**30.65**	0.904	150.56
DynCodec(QP_20, FD_3)	30.37	0.902	**133.87**

4.2.1 *Comparison of performance on the DyNeRF Dataset.*

Table 1 compares DynCodec with K-Planes and NeRFPlayer in terms of PSNR, SSIM, and per-frame model size. The results for K-Planes and NeRFPlayer are taken from [18]. DynCodec achieves similar reconstruction quality while using much smaller model sizes. At QP_10 with FD_3, DynCodec reaches higher quality than K-Planes, while requiring much less storage. Compared to NeRFPlayer, DynCodec provides similar visual quality but reduces the model size significantly.

Fig. 2 presents qualitative comparisons on the *flame salmon* scene. DynCodec better preserves overall scene structure and key object boundaries, particularly around the cooking region and flame. At QP_10, FD_3 maintains relatively sharp edges and consistent appearance, while at QP_20, slight blurring and loss of fine texture can be observed, especially in dynamic regions such as the person's motion and background

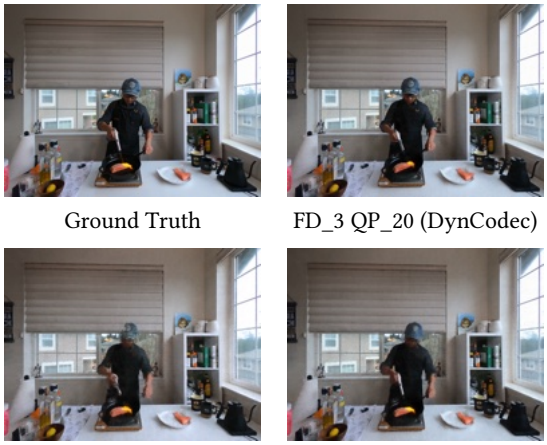

Figure 2: Qualitative comparisons on the *Flame Salmon* scene from the DyNeRF dataset.

details. This degradation is expected under stronger compression and reflects the trade-off between compact representation and reconstruction fidelity. DynCodec preserves the global structure and salient visual content, avoiding severe artifacts such as structural distortion or collapse.

Table 4 analyzes the impact of feature dimension under different compression levels. At low compression levels (QP_0 and QP_10), FD_3 consistently achieves higher PSNR and SSIM than FD_1, indicating stronger representation capacity. However, as compression increases, FD_3 becomes more sensitive to quantization, resulting in a faster quality drop compared to FD_1. In contrast, FD_1 degrades more gradually, showing greater robustness under aggressive compression. Across all scenes, both PSNR and SSIM decrease as QP increases, with a noticeable degradation beyond QP_30. Based on this trend, QP_10 and QP_20 provides the best trade-off between compression and visual quality. As the result, FD_3 is preferable when high fidelity is required, while FD_1 offers more stable performance in highly compressed settings.

These results show that DynCodec offers flexible control over the trade-off between storage and quality. FD_3 is preferable when higher rendering quality is desired, while FD_1 provides more stable performance at high compression rates. QP_10 and QP_20 levels provide an effective balance between fidelity and efficiency, while higher QP levels reduce storage cost in more resource-constrained scenarios.

4.2.2 *Extended Evaluation on Dynamic Synthetic dataset.*

Table 5 presents a comparative analysis of FD_1 and FD_3 across different quantization parameter levels by evaluating rendering quality using PSNR and storage efficiency based on model size. 'FD_1' describes the feature dimension with 1, while 'FD_3' represents the feature dimension with

Table 2: Comparison of FD_1 and FD_3 across different QP levels on the DyNeRF dataset. PSNR (dB) and SSIM are averaged out over the number of frames.

QP	Feature Dimension factor 1 (FD_1)											
	Coffee Martini		*Cook Spinach*		*Cut Roasted Beef*		*Flame Salmon*		*Flame Steak*		*Sear Steak*	
	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM
Original	29.07	0.8767	32.31	0.9214	32.47	0.9222	29.66	0.8856	32.97	0.9296	33.19	0.9301
QP_0	27.90	0.8756	30.17	0.9076	30.22	0.9087	28.30	0.8756	31.03	0.9123	31.91	0.9136
QP_10	27.84	0.8756	30.14	0.9076	30.20	0.9086	28.26	0.8756	30.87	0.9106	31.26	0.9114
QP_20	27.32	0.8753	30.06	0.9011	29.90	0.9071	28.22	0.8753	30.63	0.9105	30.88	0.9096
QP	**Feature Dimension factor 3 (FD_3)**											
Original	29.19	0.8782	32.50	0.9229	32.66	0.9234	30.20	0.9011	33.12	0.9311	33.22	0.9298
QP_0	28.15	0.8775	31.17	0.918	31.64	0.9153	29.01	0.8814	32.53	0.9295	32.51	0.9203
QP_10	28.10	0.8774	31.14	0.9178	31.61	0.9152	28.42	0.8789	32.43	0. 9295	32.19	0.9203
QP_20	27.92	0.877	30.90	0.9169	31.49	0.9146	28.34	0.8760	32.12	0.9292	31.47	0.9121

Table 3: Average PSNR and per-frame model size of FD_1 and FD_3 across four synthetic scenes. "QP Level" indicates the different compression rates applied. "Original" represents the result without any compression.

	FD_1 (PSNR in dB, Size in KB/MB)					FD_3 (PSNR in dB, Size in KB/MB)				
QP Level	Lego	Amily	Pig	Worker	Size	Lego	Amily	Pig	Worker	Size
Original	34.58	44.78	37.63	41.50	10.41MB	35.88	46.49	38.01	43.60	10.61MB
qp_0	33.68	43.83	37.07	41.17	725.4KB	34.98	45.89	37.74	42.72	2.09MB
qp_10	33.24	43.66	36.89	41.15	415.24KB	34.79	45.47	37.12	42.47	1.10MB
qp_20	31.01	38.92	34.84	36.08	184.65KB	32.03	40.81	35.79	38.74	448.53KB

3. The "Original " row represents the uncompressed model, while `qp_0` to `qp_20` indicate increasing levels of compression. The results highlight the trade-off between compression efficiency and visual fidelity, showing how different QP settings impact performance. At higher QP levels (QP_30 and QP_40), model size becomes extremely compact, but at the cost of significant quality degradation. These results are shown in the supplementary results. Additionally, we also presented qualitative comparisons of the re-rendering results, including RGB outputs and extracted depth maps, shown in the supplementary materials.

The original models achieve the highest PSNR quantitatively, with FD_3 consistently outperforming FD_1 across all scenes. As QP increases, PSNR progressively decreases, indicating a loss in rendering quality. The drop becomes more significant at QP_20, when FD_1 experiences a clear decline compared to FD_3.

These results highlight a trade-off between compression and quality, where QP_10 and QP_20 offer the best balance, maintaining PSNR above 35 dB while significantly reducing storage. These results demonstrate that DynCodec is effective for dynamic scene compression and can be applied in practical settings, including resource-limited and real-time immersive systems.

5 Conclusion

We presented DynCodec, a codec-compatible framework for compressing dynamic neural scene representations. By modeling dynamic content as compact residual features over a shared static base and integrating factorized representation with video codec-based compression, DynCodec significantly reduces model size while maintaining competitive rendering quality. These results highlight the potential of combining neural representations with standard video codecs for efficient dynamic scene delivery. Future work will focus on improving robustness under aggressive compression, optimizing decoding and rendering efficiency. In conclusion, this work demonstrates a practical step toward integrating neural scene representations into bandwidth-constrained immersive systems.

Acknowledgments

We thank the anonymous reviewers for their constructive comments. This work was supported in part by NSF under grant CNS-2200042.

References

[1] Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. 2021. Mip-NeRF: A Multiscale Representation for Anti-Aliasing Neural Radiance Fields. *arXiv preprint arXiv:2103.13415* (2021).

[2] Eric R. Chan, Connor Z. Lin, Matthew A. Chan, Koki Nagano, Boxiao Pan, Shalini De Mello, Orazio Gallo, Leonidas J. Guibas, Jonathan Tremblay, Sameh Khamis, and Gordon Wetzstein. 2022. Efficient Geometry-Aware 3D Generative Adversarial Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[3] Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. 2022. TensoRF: Tensorial Radiance Fields. In *European Conference on Computer Vision (ECCV)*.

[4] Sara Fridovich-Keil, Giacomo Meanti, Frederik Warburg, Benjamin Recht, and Angjoo Kanazawa. 2023. K-Planes: Explicit Radiance Fields in Space, Time, and Appearance. arXiv:2301.10241 [cs.CV]

[5] Alain Hore and Djemel Ziou. 2010. Image quality metrics: PSNR vs. SSIM. In *2010 20th international conference on pattern recognition*. IEEE, 2366–2369.

[6] Wenbo Hu, Yuling Wang, Lin Ma, Bangbang Yang, Lin Gao, Xiao Liu, and Yuewen Ma. 2023. Tri-MipRF: Tri-Mip Representation for Efficient Anti-Aliasing Neural Radiance Fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

[7] Andrei Khakhulin, Evgeny Burnaev, and Victor Lempitsky. 2023. HexPlane: A Volumetric Representation for Neural Rendering. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

[8] Tianye Li, Mira Slavcheva, Michael Zollhöfer, Simon Green, Christoph Lassner, Changil Kim, Tanner Schmidt, Steven Lovegrove, Michael Goesele, Richard Newcombe, and Zhaoyang Lv. 2022. Neural 3D Video Synthesis from Multi-view Video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 5521–5531.

[9] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. 2020. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 405–421.

[10] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. 2022. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. *ACM Trans. Graph. (SIGGRAPH)* (2022).

[11] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. 2020. D-NeRF: Neural Radiance Fields for Dynamic Scenes. arXiv:2011.13961 [cs.CV]

[12] Clemens Reiser, Songyou Peng, Yiyi Liao, Stephen Lombardi, Boxin Shi, Taku Komura, Gerard Pons-Moll, and Andreas Geiger. 2021. KiloNeRF: Speeding up Neural Radiance Fields with Thousands of Tiny MLPs. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 14335–14345.

[13] Liangchen Song, Anpei Chen, Zhong Li, Zhang Chen, Lele Chen, Junsong Yuan, Yi Xu, and Andreas Geiger. 2023. NeRFPlayer: A Streamable Dynamic Scene Representation with Decomposed Neural Radiance Fields. arXiv:2210.15947 [cs.CV] https://arxiv.org/abs/2210.15947

[14] Ayush Tewari, Justus Thies, Ben Mildenhall, Pratul Srinivasan, Edgar Tretschk, Yifan Wang, Christoph Lassner, Vincent Sitzmann, Ricardo Martin-Brualla, Stephen Lombardi, Tomas Simon, Christian Theobalt, Matthias Niessner, Jonathan T. Barron, Gordon Wetzstein, Michael Zollhoefer, and Vladislav Golyanik. 2022. Advances in Neural Rendering. arXiv:2111.05849 [cs.GR] https://arxiv.org/abs/2111.05849

[15] Suramya Tomar. 2006. Converting video formats with FFmpeg. *Linux journal* 2006, 146 (2006), 10.

[16] Edgar Tretschk, Ayush Tewari, Vladislav Golyanik, Michael Zollhoefer, Christoph Lassner, and Christian Theobalt. 2021. Non-Rigid Neural Radiance Fields: Reconstruction and Novel View Synthesis of a Dynamic Scene from Monocular Video. In *ICCV*.

[17] Zirui Wang, Shangzhe Wu, Weidi Xie, Min Chen, and Victor Adrian Prisacariu. 2021. NeRF−−: Neural radiance fields without known camera parameters. *arXiv preprint arXiv:2102.07064* (2021).

[18] Minye Wu, Zehao Wang, Georgios Kouros, and Tinne Tuytelaars. 2023. TeTriRF: Temporal Tri-Plane Radiance Fields for Efficient Free-Viewpoint Video. arXiv:2312.06713 [cs.CV] https://arxiv.org/abs/2312.06713

[19] Alex Yu, Ruilong Ye, Matthew Tancik, Angjoo Kanazawa, and Jitendra Malik. 2021. PlenOctrees for Real-time Rendering of Neural Radiance Fields. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (2021).

[20] Jin Zhou, Mufeng Zhu, Yao Liu, and Songqing Chen. 2025. NeRF-Compressor: Enhancing Dynamic Scene Representation for Efficient 6-DoF Object Transportation. In *IEEE 27th International Workshop on Multimedia Signal Processing (MMSP)*.

[21] Mufeng Zhu, Yuan-Chun Sun, Na Li, Jin Zhou, Songqing Chen, Cheng-Hsin Hsu, and Yao Liu. 2024. Dynamic 6-DoF Volumetric Video Generation: Software Toolkit and Dataset. In *2024 IEEE 26th International Workshop on Multimedia Signal Processing (MMSP)*. 1–6. doi:10.1109/MMSP61759.2024.10743552